Next-Day AQI Forecasting

Decision document: data preparation and feature engineering

Scope	Every decision taken during exploratory data analysis (EDA), with the evidence behind it
Source	`notebooks/eda.ipynb` and `notebooks/utility.py`, run top to bottom on `data/processed/aqi_weather_2015_2025.csv`
Audience	Technical team building and reviewing the model
Status	Proposed. Items marked *test on validation* are settled in the modelling notebook.
Date	21 October 2026

1. Summary

The model predicts **tomorrow's air-quality category** for 14 Indian cities from what is known today. Before modelling, the data was checked for errors, gaps and traps, and each feature was tested for whether it actually helps. This document lists the twelve decisions that came out of that work. Each one has a plain-language reason, the measured proof, and a concrete example. All numbers are reproducible from the notebook.

#	Decision	Main evidence
D1	Build every category from the AQI **number**, never the text label	56 labels disagree with their number; 51 sit exactly on a boundary (50, 100, …)
D2	Target = tomorrow's category for the same city; the target is never filled	Avoids learning from invented answers
D3	Fill AQI gaps of up to 3 days with the last known value; drop longer gaps	Copies older than 3 days are worse than a plain city + month average; the limit closes 90% of gaps
D4	One-hot encode city; re-test it once lag features exist	Average AQI 75 in Bengaluru vs 215 in Delhi
D5	Six pollutant yes/no flags instead of 52 text combinations	PM2.5 is the main pollutant before 92% of Severe days, 38% of Good days
D6	Score with macro F1 and Very Poor / Severe recall; balanced class weights	Severe is only 2.4% of days
D7	Add month as a feature	Severe: 10.4% of November days, 0.0% of July days
D8	Keep every outlier; detect them with skew-aware fences	Plain 1.5 × IQR flags 8,427 rain days; the skew-aware rule flags 127
D9	Today's AQI is the anchor feature; expect tree models to win	rho = 0.88; weather cannot tell Very Poor from Severe
D10	Use a reduced, non-overlapping feature set for logistic regression	Same accuracy (AUC 0.964), but stable and sensible coefficients
D11	Add 7-day lag and rolling AQI features per city	Correlation with yesterday 0.86, fading to 0.66 by day 7
D12	Split by time: train 2015–2022, validate 2023, test 2024–2025	A random split would leak neighbouring days into the test set

2. The problem and the data

Goal. For each city and day, predict the category of *tomorrow's* AQI, using today's AQI, the main pollutants named in today's bulletin, today's weather and the calendar.

Data. 51,072 city-days for 14 cities, 2015-05-01 to 2025-12-31: CPCB daily AQI bulletins joined with daily weather. After cleaning (D3), 48,827 rows remain.

Categories. The six CPCB classes are ordered from best to worst:

AQI	0–50	51–100	101–200	201–300	301–400	401–500
Category	Good	Satisfactory	Moderate	Poor	Very Poor	Severe

Two things make this harder than it looks: the rare classes (Very Poor, Severe) are the ones that matter most, and because rows are consecutive days, careless preparation can let information from tomorrow leak into today. Many of the decisions below exist to prevent exactly that.

3. Decisions

D1 Build every category from the AQI number

> **Decision:** Ignore the source's text label. Derive every category from the AQI value with `aqi_to_category`.

Why

The category should follow from the number by the CPCB table. If the two disagree, the same AQI can carry two different answers, and the model learns from contradictions.

Proof

56 of about 49,000 labelled rows disagree with their own number, 51 of them in 2018. 51 of the 56 have an AQI of **exactly** 50, 100, 200, 300 or 400: the source sometimes rounds a boundary value up to the next class. At AQI = 100 alone:

AQI exactly 100, labelled as…	Days	Correct per CPCB?
Satisfactory	217	Yes (51–100)
Moderate	23	No

Example

Bengaluru, 31 Jan 2018: AQI 100, labelled *Moderate*. By the CPCB table it is *Satisfactory*. With the decision, both the input (today's category) and the target use *Satisfactory*.

D2 Target: tomorrow's category, same city, never filled

> **Decision:** `tomorrow_status = aqi_to_category(df.groupby('city')['aqi_today'].shift(-1))`, built **before** any gap filling. Rows without a real next-day AQI are dropped.

How it works

Within each city, the AQI column is moved up one row (`shift(-1)`), so each day receives the next day's AQI. That number is then turned into its CPCB category.

Date (Delhi)	AQI today	AQI tomorrow	tomorrow_status
31 Oct 2023	359	364	Very Poor
1 Nov 2023	364	392	Very Poor
2 Nov 2023	392	468	**Severe**

The 2 November row now says: *"today AQI is 392; tomorrow will be Severe"*. That pair is what the model learns from.

Rules

Rule	What goes wrong without it
Shift **within each city**	Bengaluru's last day would get Chennai's first day as its "tomorrow".
Use the AQI **number** (D1)	Identical AQI values would get different answers.
Build the target **before** filling gaps	A filled (invented) value would become the answer for the day before it. Inputs may be filled; the target never is.
Drop rows with no real tomorrow	The last day of each city and the day before a long gap have no correct answer to learn from.

D3 Missing AQI: fill up to 3 days, drop the rest

> **Decision:** Fill gaps of up to 3 days with the last known value from the same city (`ffill_aqi(df, limit=3)`). Mark filled days with `aqi_imputed = 1`. Drop days in longer gaps.

Why forward fill

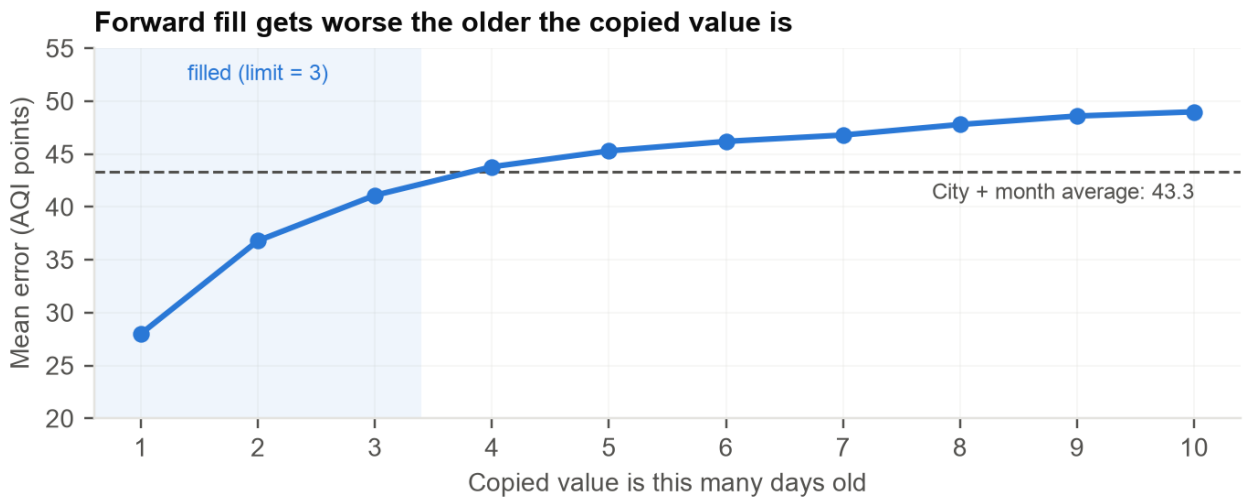
4.2% of days have no bulletin. Six methods were compared by hiding 3,000 known AQI values and filling them back:

Method	Mean error (AQI points)	Category correct
Fill with 0	154	8%
Global mean	83	32%
City mean	67	37%
City and month mean	43	52%
Forward fill (last known value, same city)	**29**	**68%**
Linear interpolation	22	74%, but it uses tomorrow's value: **leakage**

Interpolation scores best only because it peeks at the next known day, which the model will never have at prediction time. Forward fill is the best method that uses only the past.

Why the limit is 3 days

Gaps are mostly short: 903 gaps in total, 577 of them a single day, 90% are 3 days or shorter, and the longest is 177 days. But a copied value gets worse the older it is. Measured on every known day:



Mean error of a copied value by its age, against the city + month average on the same days (43.3 points, 52.5% category correct).

Copied from	1 day	2 days	3 days	4 days	5 days	7 days
Mean error	28.0	36.8	41.1	43.8	45.3	46.8
Category correct	68.2%	59.8%	55.9%	53.5%	52.4%	51.3%

Up to 3 days old, the copy beats the city + month average (41.1 vs 43.3 points). From 4 days it is worse (43.8). On category accuracy it stays slightly ahead until about 5 days. **Limit = 3 is the point where a copied value stops being better than a generic guess**; 2 or 4 would also be defensible.

Fill limit	Missing days filled	Gaps fully closed
1	903 (42%)	64%
2	1,229 (57%)	81%
3	1,397 (65%)	90%
5	1,554 (73%)	95%
7	1,636 (77%)	97%

Going from 3 to 7 closes only 7% more gaps, and the values it adds are the poor-quality old ones.

What is dropped, and what it costs

- 1,397 days are filled. 741 days (1.5%) remain in long gaps and are dropped. Only **93** of them had a known tomorrow, so that is the real loss of training examples.
- Together with rows that have no real next-day AQI, 2,245 rows (4.4%) are removed, leaving 48,827. The class balance hardly changes, and the dropped days are spread evenly across months.
- `number_of_monitoring_stations` is also dropped: no relationship with AQI (correlation ≈ 0) and missing whenever a day is filled.
- **Watch:** 181 of the dropped days fall in 2025, inside the test period, mostly in Faridabad, Kolkata, Gurugram and Patna. Report test scores per city with the number of days.

D4 City: one-hot encode, then re-test

> **Decision:** Keep `city` as 14 yes/no columns. Once lag features exist, train with and without it and keep it only if validation improves. *Test on validation.*

Why keep it

Cities sit at very different pollution levels, driven by traffic, industry, crop burning and geography, none of which the dataset measures. The city name stands in for all of it.

City	Average AQI	City	Average AQI
Bengaluru	75.2	Delhi	214.8
Chennai	84.5	Ghaziabad	212.3
Hyderabad	89.4	Greater Noida	206.5

Delhi NCR cities spend about 45–50% of days at Poor or worse; Bengaluru, Chennai and Hyderabad almost never do.

Why one-hot, and why re-test

- Label encoding (Bengaluru = 0, Delhi = 3, …) would tell the model that cities have an order or a size. One-hot gives each city its own 0/1 column, with no false order.
- Once `aqi_roll7_mean` exists (D11), a row already says "this place has been at 300 all week", which may make the city redundant. Dropping it would mean 14 fewer columns, less memorising ("Delhi → Poor"), and a model that can score a new city.

D5 Pollutants: six yes/no flags

> **Decision:** Split `prominent_pollutants` into one 0/1 flag per pollutant (`has_pm25`, `has_pm10`, …). Drop `has_nh3` (14 days only).

Why

The field holds 52 distinct strings, which are combinations of 7 pollutants (about half of all days list two or three). One-hot encoding the raw strings would create many rare columns that mean almost the same thing.

Proof

The flags carry information the AQI number alone does not: PM2.5 is the main pollutant on **91.8%** of the days before a Severe day, against **38.2%** before a Good day. Gases (CO, O_3, NO_2) mostly appear before clean days: when particles are low, a gas becomes the "worst" pollutant.

D6 Class imbalance: measure and weight for the rare classes

> **Decision:** Evaluate with **macro F1** plus **recall on Very Poor and Severe**. Train with `class_weight='balanced'`.

Proof

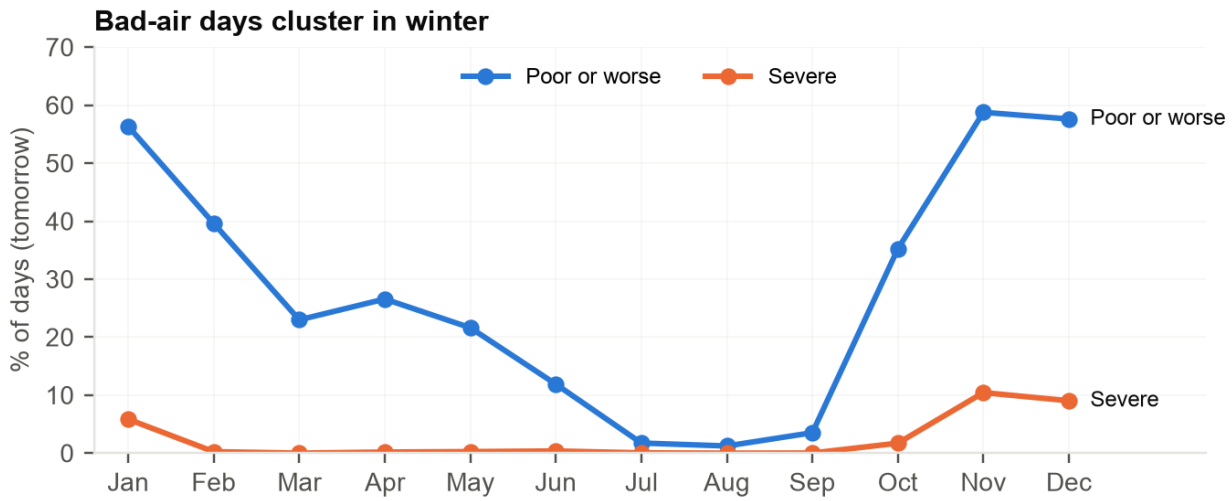
	Good	Satisfactory	Moderate	Poor	Very Poor	Severe
Days	4,037	16,000	15,134	7,874	4,604	1,178
Share	8.3%	32.8%	31.0%	16.1%	9.4%	2.4%

Why

Satisfactory has about 13.6 times as many days as Severe. A model that never predicts Severe would still score high accuracy, while missing exactly the days that matter. Macro F1 gives every class equal weight. Balanced weights make each Severe day count about 7 times more in training, and each Satisfactory day about 0.5.

D7 Season: add month

> **Decision:** Add `month` (1–12) as a feature; consider a winter or stubble-burning flag in modelling.



Share of days whose next day is Poor or worse, and Severe, by month (all cities, 2015–2025).

July is clean: 59.2% of days are Satisfactory and 0.0% are Severe. November is the worst: 10.4% Severe and about 59% Poor or worse. Air quality turns in October (crop burning, cooler and calmer nights). Severe days occur almost only in winter, so Severe recall will be the hardest metric.

D8 Outliers: detect with skew-aware fences, remove none

> **Decision:** Use `detect_outliers` (utility.py) instead of the plain 1.5 × IQR rule. Keep every value; all are physically plausible. For logistic regression only: `log1p(precip_sum)` (or a rained yes/no flag) and pressure relative to each city's normal.

Why the plain rule is not enough

The 1.5 × IQR rule assumes a roughly symmetric distribution. On a skewed feature it flags ordinary values in the long tail. The new function checks the skew first: roughly symmetric features get the plain rule; heavily skewed ones (|skew| > 1) get the *adjusted boxplot*, whose fences widen on the long side. A value that covers half the data (0 mm of rain) is treated as a spike, never as an outlier. An optional per-group mode judges each value against its own city.

Proof

Feature	Plain 1.5 × IQR	Skew-aware	What the plain rule was really flagging
precip_sum	8,427	127	Every rainy day: 57% of days have 0 mm, so the fences collapse around zero
pressure_mean	3,849	16 (by city)	Every single Bengaluru day (3,848 of 3,848): the city sits at high altitude, so its pressure is lower
wind_speed_mean	1,490	609	Normal windy days in a right-skewed feature
aqi_today	548	548	Unchanged: skew is ≈ 1, so the plain rule applies

Why nothing is removed

Every extreme value is physically possible (humidity 10–98%, temperature 1–47 °C, AQI 14–500, rain up to 251 mm). The highest-AQI days are real pollution episodes, and they are exactly the Severe days the model most needs to learn. **An extreme value is not the same as a wrong value.** Tree models need no transformation; logistic regression gets the two transforms above.

D9 Which features carry signal

> **Decision:** Keep today's AQI and the strong weather features. Expect tree models (random forest, XGBoost) to beat logistic regression, which remains the interpretable baseline.

Method

Group all days by *tomorrow's* category and compare *today's* value of each feature across the six groups. If the groups differ, the feature helps. **rho** (−1 to +1) says how strongly it moves with the class order; **shape** says whether it moves in a straight line, bends, or reverses (U-shape).

Feature	rho	Typical value, Good → Severe	Shape	Cannot tell apart
`aqi_today`	0.88	46 → 73 → 139 → 236 → 332 → 406	straight	-
`precip_sum`	-0.6	5.9 → 1.1 → 0 → 0 → 0 → 0	bends	Moderate ≈ Poor, Poor ≈ Very Poor, Very Poor ≈ Severe
`cloud_cover_mean`	-0.59	93.5 → 69.2 → 22 → 9 → 7.4 → 5.4	bends	Poor ≈ Very Poor, Very Poor ≈ Severe
`temp_min`	-0.51	25.1 → 25.1 → 22.2 → 17.5 → 12 → 12	bends	Good ≈ Satisfactory, Very Poor ≈ Severe
`humidity_mean`	-0.4	84.5 → 77.3 → 62.8 → 63 → 69.2 → 69.8	reverses	Moderate ≈ Poor, Very Poor ≈ Severe
`pressure_mean`	0.16	987.9 → 985.3 → 987.5 → 990.4 → 992.2 → 991.9	reverses	Very Poor ≈ Severe

Findings

- **Today's AQI is by far the strongest signal** (rho 0.88) and the only feature that separates all six classes.
- Cold, dry, clear, calm days come before bad air. But the weather features **bend**: they change a lot from Good to Poor, then flatten. **Weather says a bad day is coming, not how bad**: it cannot tell Very Poor from Severe.
- `humidity_mean` reverses (84 → 63 → 70), a U-shape that logistic regression cannot use but trees can.
- `pressure_mean` has almost no signal (rho 0.16); its differences come from city altitude (D8).

D10 Collinearity: a reduced feature set for logistic regression

> **Decision:** For logistic regression drop `temp_mean`, `temp_max`, `wind_speed_max`, `dew_point_mean` and `wind_dir_mean`. Test dropping `humidity_mean` too. Tree models keep all features.

The problem in plain words

When two features carry the same information, logistic regression cannot tell which one deserves the credit. Think of house area given in both square metres and square feet: any split of the weight gives the same predictions, so the model picks one almost at random, and a slightly different dataset makes it pick another. The predictions stay good, but the model's *explanation* (its coefficients) can no longer be trusted. Here, `temp_range` is exactly `temp_max` − `temp_min`.

How the overlap is measured: VIF

VIF asks, for each feature, *"how well can the other features predict this one?"* About 1 means unique; 5–10 is moderate; above 10 means it is mostly a copy. Computed on the training years only.

Feature	VIF: all features	VIF: reduced set
temp_max, temp_min, temp_range	≈ 10^{15} (perfect copy)	temp_min 3.25, temp_range 7.38
temp_mean	228	dropped
dew_point_mean	45.7	dropped

Feature	VIF: all features	VIF: reduced set
humidity_mean	35.3	5.24
wind_speed_mean / _max	5.55 / 5.46	1.43 / dropped
aqi_today	2.33	2.33

In the reduced set every VIF is below 10. Dropping `humidity_mean` as well brings `temp_range` from 7.38 down to 2.78.

Proof: the same model, trained on two different periods

A logistic regression was trained to answer *"will tomorrow be Poor or worse?"* on 2015–2018 and again on 2019–2022, with all features and with the reduced set. A minus sign means "more of this → less likely to be bad air".

Coefficient	All, 2015–18	All, 2019–22	Reduced, 2015–18	Reduced, 2019–22
temp_min	**+0.05**	**+0.36**	-0.27	-0.54
humidity_mean	**-0.06**	**+0.28**	-0.12	-0.36
dew_point_mean	-0.09	-0.76	dropped	dropped
aqi_today	2.54	2.81	2.54	2.8
Validation AUC	0.964	0.965	0.964	0.965

- **Accuracy is identical** (AUC 0.964–0.965), so the reduced set costs nothing.
- **With all features the explanation breaks:** `temp_min` comes out positive ("warmer nights → worse air"), the opposite of the physics in D9; `humidity_mean` flips sign between periods; `dew_point_mean` changes eightfold.
- **With the reduced set it makes sense:** `temp_min` is negative in both periods and `humidity_mean` keeps its sign. `aqi_today`, which has no copies, is stable either way.
- Tree models ask one yes/no question at a time and simply use whichever copy they meet first, so they keep all features.

D11 Lag and rolling features

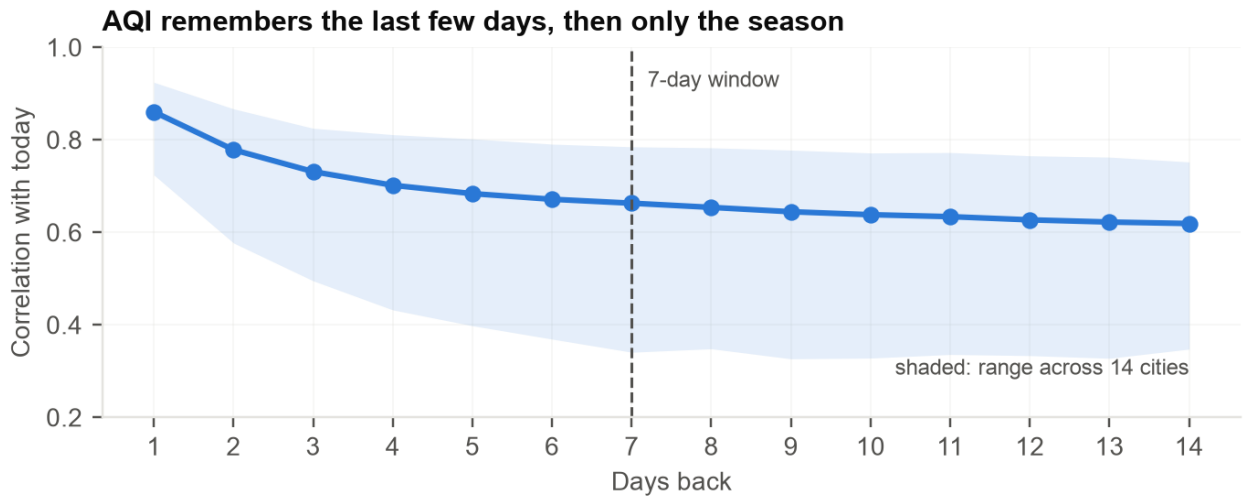
> **Decision:** Add, per city and using only today and earlier days: `aqi_roll7_mean`, `aqi_roll7_std`, `aqi_diff1`, `aqi_dev7` and `month`, via `add_trend_features`. Keep `aqi_diff1` and `aqi_dev7` only if validation improves.

The problem

The model reads one row at a time and does not know the order of the rows. From `aqi_today` alone it cannot tell whether AQI has been climbing all week or jumped only today. A person forecasting tomorrow would look at the last few days. These features copy a short summary of the recent past into today's row.

Feature	Question it answers	Delhi, 2 Nov 2023
`aqi_roll7_mean`	What has the usual level been this week? (last 7 days, today included)	336.0
`aqi_roll7_std`	Has the week been calm or jumpy?	43.5
`aqi_diff1`	Is AQI rising or falling since yesterday?	+28.0
`aqi_dev7`	How unusual is today against the previous 7 days?	+75.4
`month`	What time of year is it? (D7)	11

Why a 7-day window



Autocorrelation: correlation between a city's AQI today and its AQI k days earlier; average of 14 cities.

AQI has a strong memory: 0.86 with yesterday. It drops quickly over the first five days (0.86 → 0.68) and then hardly moves (0.66 at 7 days, 0.62 at 14). That flat tail is the season, which `month` already covers. Seven days captures the part of the memory that changes day to day.

Example: one week in Delhi

Date	AQI	roll7 mean	roll7 std	diff1	dev7	Tomorrow
27 Oct	261.0	257.7	28.4	5.0	12.7	Very Poor
28 Oct	304.0	265.7	32.7	43.0	46.3	Very Poor
29 Oct	325.0	267.4	35.8	21.0	59.3	Very Poor
30 Oct	347.0	279.4	46.5	22.0	79.6	Very Poor
31 Oct	359.0	299.3	46.6	12.0	79.6	Very Poor
01 Nov	364.0	316.6	44.6	5.0	64.7	Very Poor
02 Nov	392.0	336.0	43.5	28.0	75.4	Severe

AQI rose every day; the weekly mean trails behind today's value (336 vs 392); today is 75 points above the previous week. The next day was Severe. Today's AQI alone shows a high level, but not the week-long climb.

Proof: do they carry signal?

Feature	rho with tomorrow's class	Correlation with aqi_today	Reading
`aqi_roll7_mean`	0.85	0.92	Strong, but mostly a smoothed copy of today
`aqi_roll7_std`	0.61	0.53	**Most useful new feature**: clear signal, new information
`aqi_dev7`	0.22	0.37	Weak on its own
`aqi_diff1`	0.14	0.21	Weak on its own

The hard case: today Very Poor, tomorrow Severe?

Split Very Poor days by…	Group	% Severe tomorrow
AQI level today	300–340 / 340–370 / 370–400	3.1% / 10.8% / 23.8%
Direction (aqi_diff1)	falling / rising	9.8% / 9.6%
Jump vs last week (aqi_dev7)	lowest → highest quarter	10.3% / 10.7% / 9.9% / 7.9%

Honest result: the **level** decides it (3% → 24%), not the direction (about 10% either way). The biggest jumps are even slightly less likely to turn Severe, because AQI tends to fall back after a spike. The trend features are cheap and safe, but they are not the answer to Very Poor vs Severe; validation decides which ones stay.

Rules that keep the features free of leakage

- **Only the past:** every feature uses today and earlier days, never tomorrow. Tested: changing every AQI value after a cut-off date leaves all features before it unchanged.
- **Within each city:** Delhi's history never ends up in Bengaluru's row.
- **By calendar day:** "yesterday" is always the previous date. A missing day gives an empty value, never a value from further back.
- **Call after `ffill_aqi` and before dropping rows.** Calling it after the drop empties 810 `aqi_diff1` values, because a dropped row's AQI is still a valid past value for the next day.

D12 Split by time

Decision: `split_train_val_test`: train 2015–2022 (33,731 rows), validation 2023 (5,104), test 2024–2025 (9,992). The test set is used once, for the final model.

Why

Neighbouring days are very similar (correlation 0.86, D11). A random split would put 1 November in training and 2 November in the test set, so the model would be tested on days it has almost seen. Scores would look better than they will be in real use. Splitting by year means the model is always tested on a later period it has never seen.

4. Implementation

Pipeline order

The order matters: three of the steps protect against leakage only if they run in this sequence.

```
df = pd.read_csv(DATA_PATH, parse_dates=['date'])
df = df.rename(columns={'aqi_value': 'aqi_today', 'air_quality_status': 'status_today'})
df = df.sort_values(['city', 'date']).reset_index(drop=True)

# 1. Target FIRST, from the real AQI number (D1, D2)
df['tomorrow_status'] = aqi_to_category(df.groupby('city')['aqi_today'].shift(-1))

# 2. Pollutant flags has_pm25, has_pm10, ... (drop has_nh3), and drop
#    number_of_monitoring_stations (D3, D5): see eda.ipynb sections 3.2 and 5

# 3. Fill input gaps of <= 3 days; adds aqi_imputed, recomputes status_today (D3)
df = ffill_aqi(df, limit=3)

# 4. Lag / rolling features + month, BEFORE dropping rows (D7, D11)
df = add_trend_features(df)

# 5. Drop rows with no input or no real answer (D2, D3)
df = df.dropna(subset=['aqi_today', 'tomorrow_status'])

# 6. Split by year (D12); fit any scaler / encoder / city-relative pressure on df_train only
df_train, df_val, df_test = split_train_val_test(df)
```

Why this order

- **Target before filling:** otherwise filled values become answers.
- **Features before dropping:** otherwise days next to a gap lose their "yesterday".
- **Fit on training years only:** scalers, the one-hot encoder and the per-city pressure baseline must not see 2023–2025.

Helper functions (notebooks/utility.py)

Function	What it does	Decision
`aqi_to_category`	AQI number → CPCB category	D1, D2
`ffill_aqi`	Fills gaps up to a limit from the same city's past; adds aqi_imputed	D3
`split_train_val_test`	Train ≤ 2022, validation 2023, test ≥ 2024	D12
`class_signal_summary`	Direction, overlap and shape of each feature across classes	D9
`detect_outliers`	Skew-aware outlier fences, optional per city; returns summary + mask	D8
`add_trend_features`	Per-city 7-day lag and rolling features + month, past only	D7, D11

Feature sets per model

	Logistic regression (baseline)	Tree models (RF, XGBoost)
Weather	Reduced set (D10): temp_min, temp_range, humidity_mean*, precip_sum, pressure_mean, cloud_cover_mean, wind_speed_mean, wind_u_mean, wind_v_mean	All weather features
Transforms	log1p(precip_sum); pressure relative to city training mean; standardise all	None needed
AQI history	aqi_today or aqi_roll7_mean (r = 0.92), not both; aqi_roll7_std	All trend features
Other	month, pollutant flags, aqi_imputed, city one-hot*	Same
Missing values	Drop rows with empty trend features	Handled natively

* to be tested on validation.

5. Open items for the modelling phase

#	Item	Why
1	Score the persistence baseline (tomorrow = today) first	Sets the bar every model must beat
2	Compare with and without city (D4)	Lag features may already carry the city level
3	Test dropping humidity_mean for logistic regression (D10)	Removes the last moderate VIF
4	Keep aqi_diff1 / aqi_dev7 only if validation improves (D11)	Weak on their own
5	Re-run VIF with the trend features	aqi_roll7_mean correlates 0.92 with aqi_today
6	Check drift between train, validation and test years	Class mix by year, the 2020 lockdown dip, cities that start in 2017–18
7	Report test results per city with day counts	181 test-period days were dropped (D3)

Appendix: terms used

Term	Meaning in this document
Leakage	Information from the future (tomorrow) reaching the model's inputs. It makes scores look better than real use.
Forward fill	Filling a missing day with the last known value of the same city.
Autocorrelation	Correlation between a series and itself k days earlier: how much AQI "remembers".
rho (Spearman)	Correlation from −1 to +1 that uses only the order of values; suits ordered classes like Good → Severe.
IQR, 1.5 × IQR rule	IQR is the range of the middle 50% of values. The rule flags values more than 1.5 IQR outside it.
Skew	How lopsided a distribution is. |skew| < 0.5 symmetric, 0.5–1 moderate, > 1 heavy.
Adjusted boxplot	Outlier fences that widen on the long side of a skewed feature (uses the medcouple, a robust skew measure).
VIF	How well the other features predict this one. About 1 unique, above 10 mostly a copy.
One-hot encoding	One 0/1 column per category, so no false order is implied.
Macro F1	F1 score averaged over classes with equal weight, so rare classes count as much as common ones.
Recall	Of the days that really were Severe, the share the model caught.
AUC	How well a model ranks positive cases above negative ones; 0.5 is chance, 1.0 is perfect.